



GitHub Agentic Workflows

Repository Automation with AI Agents

Write repository automation in natural language, execute with security-first AI agents

45-60 min · Developers · DevOps · Platform Engineers · Engineering Managers

"How do I automate repository tasks that require judgment — without writing complex YAML workflows?"

Manual triage, repetitive reviews, and hand-assembled reports cost every team hours per week.
Describe the outcome in Markdown. Compile it. Let AI navigate the path.

Developers

Automate triage, reviews, and CI diagnosis expressed in plain English



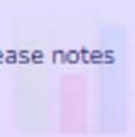
DevOps & Platform

Replace brittle YAML pipelines with adaptive, security-first agent workflows



Engineering Managers

Get weekly health reports and release notes without anyone assembling them



30x

reduction in workflow definition length — 8-line Markdown vs. 240-line YAML

100+

workflows running continuously in Peli's Agent Factory on github/gh-aw

0

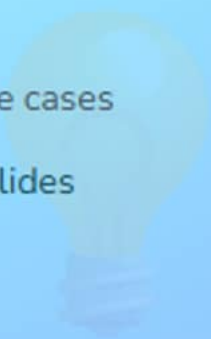
direct writes — AI reads everything, validation pipeline executes mutations

PART 1

Why It Matters

Three paths, three flagship use cases

Find your path in the first 12 slides

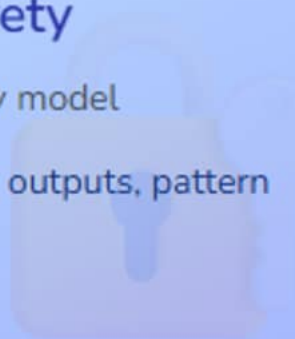


PART 2

Architecture & Safety

Compiler-enforced security model

Three-phase pipeline, safe outputs, pattern vocabulary



PART 3

Patterns by Path

10 use cases across a 3x3 matrix

Find your cell — see which peers shipped it

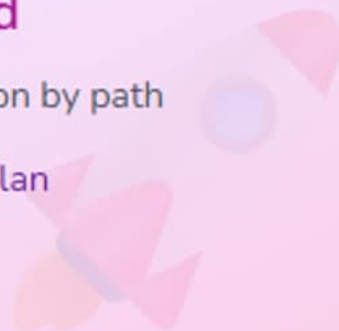


PART 4

Your Path Forward

Week-by-week progression by path

Monday-morning action plan



Click any section to jump directly there

PART 1

Why Agentic Workflows Matter

Choose your path — A, B, or C — and see yourself in the first 12 slides



Path A: Triage at Scale

High-volume, low-judgment work —
issues, labels, PR first-pass



Path B: Visibility & Synthesis

Async reporting, health dashboards,
release notes, research



Path C: Orchestration

Multi-agent workflows, worker
dispatch, ChatOps commands

Traditional CI/CD: write YAML, debug YAML, maintain YAML forever
↳ Agentic: write intent → compile → AI adapts to context

YAML Maintenance vs. Markdown Intent

Same outcome — one requires a workflow engineer, one does not

✗ Traditional GitHub Actions

- 1 Write explicit YAML steps
Every branch, conditional, and API call by hand
- 2 Debug step failures
Log dive required for every edge case
- 3 New edge case appears
Add another conditional branch and redeploy
- 4 Requirement changes
Rewrite the step sequence from scratch



Months of YAML maintenance ahead

✓ GitHub Agentic Workflows

- 1 Write intent in Markdown
Analyze new issues and apply labels — done
- 2 `gh aw compile`
Compiler generates the secure YAML lock file
- 3 AI adapts to context
Agent reads repository state and decides
- 4 Edit the Markdown body
Only frontmatter changes need recompile



Minutes to intent, seconds to compile

240-line YAML → 8-line Markdown intent

Which Problem Is Yours?



Path A: Triage at Scale

Issue Triage — on issues.opened, applies 1-3 labels with a comment explaining the decision.

Pattern: Reactive → Labels

Safe outputs: add-labels, add-comment

15 minutes to ship



Path B: Visibility & Synthesis

Weekly Health Report — scheduled, synthesizes all repository activity into an actionable issue.

Pattern: Synthesis → Issue

Safe outputs: create-issue (close-older)

15 minutes to ship



Path C: Orchestration

Multi-Agent Orchestration — dispatches parallel workers and synthesizes a coordinated summary.

Pattern: Orchestrated → Aggregated

Safe outputs: dispatch-workflow

1 day to ship

Zero to Deployed: Under Five Minutes

Write intent, compile to secure YAML, push — done

```
$ gh aw new issue-triage
Created .github/workflows/issue-triage.md
Edit the Markdown body with your automation intent

> "# edited: added intent and safe-outputs in frontmatter"
$ gh aw compile issue-triage
🔒 Generating secure lock file ...
Generated .github/workflows/issue-triage.lock.yml
Agent job: read-only sandbox (ubuntu-slim)
Safe-output handlers: add-labels (max 3), add-comment (max 1)

✓ git add .github/workflows/ && git push
✓ Runs on every new issue — zero YAML written by hand
```

gh aw new → compile → push — first workflow in minutes

Four Shifts That Change Everything



Intent over Steps

Describe what should happen — AI figures out the path. No conditionals, no edge-case branching.



Adaptive over Deterministic

Agent reads repository context and decides. Same intent, smarter execution every run.



Read-only by Default

Agent has zero write access. Trust is built through restriction — safe outputs are the write path.



Specialized over Monolithic

Many focused workflows outperform one do-everything agent. Build a factory, not a monolith.

PART 2

Architecture & Safe Execution

The compiler enforces security — not author discipline



Author in Markdown

Intent body + frontmatter triggers and permissions



Compile to Lock File

gh aw compile generates tamper-proof secure YAML



Execute in Sandbox

Read-only agent + validated safe-output handlers

Security enforced at compile time – not at runtime
↳ Compiler = policy. Lock file = proof.

Author → Compile → Execute

Every agentic workflow passes through three distinct phases



The lock file is a generated artifact — never edit it directly. Edit the .md source and recompile.

A Workflow in Its Natural State

.github/workflows/issue-triage.md

```
---
on:
  issues:
    types: [opened]
permissions:
  contents: read
  issues: read
tools:
  github:
    toolsets: [issues, labels]
safe-outputs:
  add-labels:
    allowed: [bug, feature, docs]
    max: 3
  add-comment:
    max: 1
---
```

Issue Triage

Analyze new issues and apply



Frontmatter = security contract

Permissions and safe-outputs declared before any instruction — security is structural



Tools declaration

Only the GitHub APIs the agent needs — principle of least privilege



Markdown body = agent instructions

Natural language — no step sequences, no conditionals, no error handling

Why AI Cannot Write Directly — And That's the Point

✗ The Threat

Prompt injection risk

Malicious issue content could instruct AI to push code or delete branches

Unchecked write access

Direct API calls bypass audit trails and permission scoping

One compromised run = unauthorized repository modifications

💡 Safe Outputs

AI produces structured JSON output
— never executes writes directly

Validation pipeline

Schema check → sanitize → rate limit
→ permission check → audit

Scoped handler jobs execute only the approved, validated operations

✓ The Guarantee

Zero direct writes from the AI agent
in any workflow run

Every action auditable via hidden
workflow-id markers in created items

Security by compiler

Author discipline not required — the
lock file is the policy

0

unchecked writes in any run

Five Mandatory Gates — Every Safe Output Passes All

No agent request reaches GitHub without clearing this pipeline

Schema

Is the JSON output correctly structured for the safe-output type?

Format gate

Permission

Is this safe-output type declared in the workflow frontmatter?

Auth gate

Sanitize

Escape XML, validate URLs, strip injection attempts

Safety gate

Rate Limit

Is the op count within the declared max for this run?

Control gate

Audit

Embed hidden gh-aw-workflow-id markers — every item searchable

Trace gate

Noop is the sixth implicit gate: every workflow must call noop when no action is needed — or the run fails silently.

Five Pattern Families — Your Bridge to Section 3

Every use case in this talk maps to one of these patterns

Continuous

Scheduled improvements — simplification, docs, dependency upgrades

`schedule:`

Reactive

Event-triggered analysis — CI failure, PR opened, new issue

`on: event`

Synthesis

Aggregate and report repository state — health, release notes, digest

`schedule:`

Interactive

Comment command triggers — slash commands invoke in-thread responses

`issue_comment`

Orchestrated

Multi-agent dispatch and synthesis — parallel workers, summary report

`dispatch-workflow`

Pattern = trigger type + safe-output type. Once you know the pattern, the workflow almost writes itself.

Two Comment Trigger Mechanisms

Comment Workflows (gh-aw)

Repo-owned Markdown

Version-controlled, compiled to Actions via gh aw compile

Safe-output governed

AI reads comment → validation pipeline → scoped handler

Slash commands by convention

Your workflow Markdown defines what triggers what

Audit trail: hidden gh-aw-workflow-id markers in every created item

Cloud-Agent Automations

Configured in Agents > Automations

Repository settings — not version-controlled Markdown

Trigger text delegates agent tasks

Configured text in issue or PR comment initiates work

Requires eligible Copilot plan

Business/Enterprise admin must enable the policy

Initiates documentation, investigation, or follow-up — not gh-aw slash commands

Patterns by Path & Maturity

Find your cell — see which peers have already shipped it



Path A: Triage at Scale

L1: Issue Triage → L2: PR Review → L3:
Dependency Audit



Path B: Visibility & Synthesis

L1: Health Report → L2: Release Notes
→ L3: Research Digest



Path C: Orchestration

L1: CI Failure → L2: Comment
Workflows → L3: Multi-Agent

100+ workflows in Peli's Agent Factory – validated in continuous
production

↳ Specialize, don't monolith. Build many focused workflows.

Path A: From Labels to Judgment



L1 — Issue Triage

on: issues.opened → read →
apply 1-3 labels from allowed
list → comment with reasoning.

Safe outputs: add-labels, add-
comment

15 minutes to ship

Failure mode: hitting max (raise
max: 3+)



L2 — Automated PR Review

on: pull_request.opened →
analyze diff → post inline review
comments → submit COMMENT
review.

Safe outputs: review-comment,
submit-review

1 hour to ship

Failure mode: noop if diff is empty



L3 — Dependency Audit

schedule: weekly → audit
packages → group by risk → one
contextual PR per upgrade
decision.

Safe outputs: create-pull-request
(draft, max: 3)

1 day to ship

Failure mode: noop if no upgrades
found

Path B: From Reports to Intelligence



L1 — Weekly Health Report

schedule: weekly → synthesize issues, PRs, activity → create summary issue (close-older: true).

Safe outputs: create-issue (close-older)

15 minutes to ship

Failure mode: low-signal noop spam



L2 — Release Notes

on: release.created → read merged PRs → categorize changes → draft contextual notes for team review.

Safe outputs: create-issue for team review

1 hour to ship

Failure mode: noop if no merged PRs



L3 — Research Digest

schedule: weekly → fetch HN, arXiv, GitHub Trending → filter for relevance → post curated digest.

Safe outputs: create-issue (expires: 8)

1 day to ship

Failure mode: noop if fewer than 3 relevant items

Path C: From Reaction to Coordination



L1 — CI Failure Diagnosis

on: workflow_run (failure) →
analyze logs → create structured
issue with root cause and next
steps.

Safe outputs: create-issue (close-
older)

15 minutes to ship

Failure mode: noop if not a known
failure type



L2 — Comment Workflows (gh-aw)

Compiled to Actions → on
issue_comment.created, AI
reads, safe-output handler
responds in-thread.

Safe outputs: add-comment, add-
labels

1 hour to ship

Failure mode: noop if no command
recognized



L3 — Multi-Agent Orchestration

schedule: daily → dispatch
worker workflows → collect
results → synthesize coordinated
report.

Safe outputs: dispatch-workflow,
create-issue

1 day to ship

Failure mode: worker
synchronization races

PART 4

Your Path Forward

Pick your path, commit your level, ship Monday morning



Week 1: Level 1

Deploy one read-only workflow — low risk, instant value



Week 2-3: Safe Writes

Add labels, comments, draft PRs with expiration guards



Month 2+: Level 3

Orchestration, meta-workflows, multi-agent patterns

Every factory began with one workflow and one gh aw new
↳ Start small. Specialize. Compound.

Level Up on Your Schedule

A progression contract — not a deadline

Week 1

Level 1

Deploy one read-only workflow on your chosen path — no safe writes, zero risk, instant value



Week 2-3

Safe Writes

Add safe-output writes: labels, comments, draft PRs — with max and expires guardrails in frontmatter



Week 4+

Level 2

Build a path-specific workflow: PR review, release notes, or ChatOps commands depending on your path



TARGET

Month 2+

Level 3

Orchestration, meta-workflows, research digests — advanced patterns from Peli's factory



Human-in-the-loop at every level: use draft: true for PRs and expires: 7 for issues until you trust the pattern.

Monday Morning — Path by Path

Install once, then go directly to your path

Everyone	gh extension install github/gh-aw → gh aw init → gh aw version	15 min
----------	--	--------

Path A	gh aw new issue-triage → edit body → compile → push	30 min
--------	---	--------

Path B	gh aw new weekly-health → set schedule: weekly → compile → push	30 min
--------	---	--------

Path C	gh aw new ci-diagnosis → set on: workflow_run → compile → push	30 min
--------	--	--------

Browse [githubnext/agentics](#) for ready-made templates — add any workflow with `gh aw add`.

From Scripted Steps to Adaptive Intent

Before

240-line YAML workflow with explicit steps and conditionals

Debugging requires log-diving for every edge case encountered

Requirements change: rewrite the step sequence from scratch

AI has direct write access or no automation at all

After

8-line Markdown intent statement — describe the outcome, not the path

gh aw compile catches errors before they reach production

Requirements change: edit the Markdown body, no recompile needed

AI reads everything, validation pipeline executes only validated writes

30×

reduction in workflow definition length

100+

workflows validated in continuous production

0

direct writes — every mutation through the validation pipeline

✓ What You Can Do Today

Today

- ❑ Install: gh extension install github/gh-aw and run gh aw version
- ❑ Pick your path: A (triage), B (reporting), or C (orchestration)
- ❑ Run gh aw new and write your first Markdown workflow in minutes

This Week

- ❑ Compile and deploy a Level 1 workflow on your chosen path
- ❑ Study gh aw logs — understand how the validation pipeline works
- ❑ Browse githubnext/agentics for templates matching your path

This Month

- ❑ Add safe writes (labels, comments, draft PRs) with expiration guardrails
- ❑ Implement an Orchestrator-Worker pattern across 2-3 specialized workflows
- ❑ Build a meta-workflow that monitors and optimizes your other workflows

🔑 Key Takeaway

Start with one workflow. Every compound improvement begins with a single gh aw new.

 Official Documentation

[GitHub Blog: Automate repository tasks with GitHub Agentic Workflows](#)

[GitHub Agentic Workflows: Quick Start Guide](#)

[GitHub Agentic Workflows: Creating Workflows](#)

[GitHub Agentic Workflows: Safe Outputs Reference](#)

 Community & Examples

[githubnext/agentic — Sample Pack of Agentic Workflows](#)

[Josh-Ops: Getting Started with GitHub Agentic Workflows](#)

[Automating Weekly Research with GitHub Agentic Workflows](#)

[Erik Lieben: Dependency Updates That Understand Your Code](#)



GitHub Agentic Workflows

Repository automation that judges, adapts, and compounds — while you sleep.



8-line Markdown

Replaces 240-line YAML

Compiler generates the secure lock file —
author discipline not required



Zero direct writes

Compiler-enforced security

Read-only AI + validation pipeline +
scoped handler jobs at every run



15 minutes

To your first deployed workflow

gh aw new → compile → push — pick your
path and start today

Which path is yours — and what's your Level 1 workflow?